

Report – Gemini API e conversazione interattiva

Studente: Vincenzo Lembo

Corso: Cybersecurity & Ethical Hacking

Modulo: Python AI / Gemini API

Esercitazione: W7D1 – Pratica

Argomento: Utilizzo delle Gemini API e realizzazione di una conversazione interattiva

Ambiente: Kali Linux – UTM

Linguaggio: Python

Strumenti utilizzati: Visual Studio Code, Gemini API

1. Obiettivo dell'esercitazione

L'obiettivo dell'esercitazione è stato quello di creare un semplice programma Python in grado di utilizzare le API di Gemini per interagire con un modello di intelligenza artificiale.

Durante l'esercitazione ho configurato l'API Key, impostato alcuni parametri del modello, definito un System Prompt e configurato i Safety Settings. Successivamente ho modificato il programma per permettere una conversazione continua con l'intelligenza artificiale direttamente dal terminale.

2. Creazione del progetto

Ho lavorato all'interno di un progetto Python chiamato `python_ai`.

All'interno del progetto sono presenti:

- `script.py` → contiene il programma Python;
- `.env` → contiene la chiave API di Gemini;
- `requirements.txt` → contiene le librerie necessarie;
- `venv_ai` → ambiente virtuale Python utilizzato per eseguire il programma.

Per evitare di inserire direttamente la chiave API all'interno del codice, ho utilizzato il file `.env`.

Nel file ho inserito:

```
GEMINI_API_KEY=LA_MIA_API_KEY
```

3. Importazione delle librerie

All'inizio del programma ho importato le librerie necessarie:

```
from google import genai
from google.genai import types
import os
from dotenv import load_dotenv
```

`google.genai` permette di utilizzare le API di Gemini.

`types` viene utilizzato per configurare il comportamento del modello.

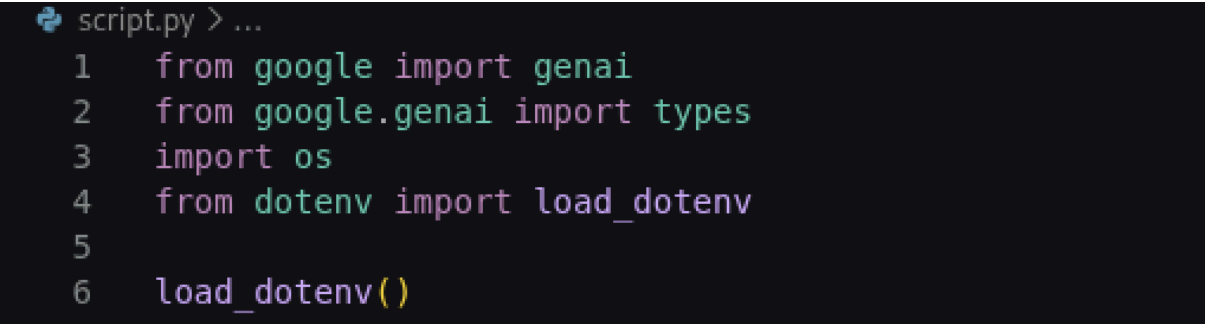
`os` permette di leggere la variabile contenente la chiave API.

`load_dotenv` permette di caricare le variabili presenti nel file `.env`.

Successivamente ho utilizzato:

```
load_dotenv()
```

per caricare le informazioni presenti nel file `.env`.



```
script.py > ...
1  from google import genai
2  from google.genai import types
3  import os
4  from dotenv import load_dotenv
5
6  load_dotenv()
```

4. Configurazione dell'API Key

Ho recuperato la chiave API tramite:

```
api_key = os.environ["GEMINI_API_KEY"]
```

Successivamente ho creato il client Gemini:

```
client = genai.Client(api_key=api_key)
```

```
8  api_key = os.environ["GEMINI_API_KEY"]
9  client = genai.Client(api_key=api_key)
```

In questo modo il programma può comunicare con il servizio Gemini utilizzando la chiave presente nel file `.env`.

5. Creazione del System Prompt

Successivamente ho creato un System Prompt per definire il comportamento dell'intelligenza artificiale:

```
system_prompt = """
Sei un esperto Cyber Security Analyst.
Analizza i dati forniti e rispondi in modo tecnico ma comprensibile.
Se non sei sicuro, dillo esplicitamente.
"""
```

```
8  api_key = os.environ["GEMINI_API_KEY"]
9  client = genai.Client(api_key=api_key)
10
11
12  system_prompt = """
13  Sei un esperto Cyber Security Analyst.
14  Analizza i dati forniti e rispondi in modo tecnico ma comprensibile.
15  Se non sei sicuro, dillo esplicitamente.
16  """
17
```

Il System Prompt serve quindi a stabilire il ruolo che deve assumere l'intelligenza artificiale.

In questo caso ho impostato Gemini come un esperto di Cyber Security, chiedendogli di fornire risposte tecniche ma allo stesso tempo comprensibili.

6. Configurazione dei Safety Settings

Ho successivamente configurato i Safety Settings:

```
safety_settings = [
    types.SafetySetting(
        category="HARM_CATEGORY_DANGEROUS_CONTENT",
        threshold="BLOCK_NONE"
    ),
    types.SafetySetting(
        category="HARM_CATEGORY_HATE_SPEECH",
        threshold="BLOCK_NONE"
    )
]
```

```

),
types.SafetySetting(
    category="HARM_CATEGORY_HARASSMENT",
    threshold="BLOCK_NONE"
),
types.SafetySetting(
    category="HARM_CATEGORY_SEXUALLY_EXPLICIT",
    threshold="BLOCK_NONE"
)
]

```

```

18  safety_settings = [
19      types.SafetySetting(
20          category="HARM_CATEGORY_DANGEROUS_CONTENT",
21          threshold="BLOCK_NONE"
22      ),
23      types.SafetySetting(
24          category="HARM_CATEGORY_HATE_SPEECH",
25          threshold="BLOCK_NONE"
26      ),
27      types.SafetySetting(
28          category="HARM_CATEGORY_HARASSMENT",
29          threshold="BLOCK_NONE"
30      ),
31      types.SafetySetting(
32          category="HARM_CATEGORY_SEXUALLY_EXPLICIT",
33          threshold="BLOCK_NONE"
34      ),
35  ]

```

Questa parte permette di configurare il comportamento dei filtri di sicurezza del modello.

7. Configurazione del modello

Ho creato la configurazione generale del modello:

```

config = types.GenerateContentConfig(
    temperature=0.2,
    max_output_tokens=1024,
    system_instruction=system_prompt,
    safety_settings=safety_settings
)

```

```
37 config = types.GenerateContentConfig(  
38     temperature=0.2,  
39     max_output_tokens=1024,  
40     system_instruction=system_prompt,  
41     safety_settings=safety_settings  
42 )
```

Sono stati utilizzati diversi parametri.

Temperature

Ho impostato:

temperature=0.2

Questo valore rende le risposte più precise e meno casuali.

Max output tokens

Ho impostato:

max_output_tokens=1024

Questo parametro stabilisce la quantità massima di testo che il modello può generare nella risposta.

System Instruction

Ho collegato il System Prompt alla configurazione tramite:

system_instruction=system_prompt

Safety Settings

Ho collegato le impostazioni di sicurezza tramite:

safety_settings=safety_settings

8. Creazione della conversazione continua

La parte principale dell'esercitazione è stata la trasformazione del programma da una singola richiesta a una conversazione continua.

Ho utilizzato:

```
while True:
```

In questo modo il programma continua a chiedere all'utente una nuova domanda.

Per acquisire la domanda ho utilizzato:

```
domanda = input("Tu: ")
```

Quindi la domanda viene inserita direttamente dal terminale.

9. Comando per uscire dalla conversazione

Per evitare che il programma continui all'infinito, ho inserito una condizione:

```
if domanda.lower() == "esci":  
    print("Arrivederci!")  
    break
```

Il comando `lower()` permette di trasformare il testo in minuscolo.

In questo modo vengono riconosciute anche varianti come:

```
esci  
ESCI  
Esci  
eSci
```

Quando viene inserita la parola `esci`, il comando:

```
break
```

interrompe il ciclo `while`.

10. Invio della domanda a Gemini

La domanda dell'utente viene inviata al modello tramite:

```
response = client.models.generate_content(
```

```
model="gemini-3.5-flash",  
contents=domanda,  
config=config  
)
```

In questa parte:

- `model` indica il modello Gemini utilizzato;
- `contents=domanda` invia al modello ciò che è stato scritto dall'utente;
- `config=config` applica tutte le impostazioni definite precedentemente.

Durante l'esercitazione ho inizialmente provato a utilizzare `gemini-2.5-flash`, ma il modello non risultava disponibile per la mia API Key e il terminale restituiva un errore `404 NOT_FOUND`.

Ho quindi utilizzato `gemini-3.5-flash`, che nel mio ambiente ha funzionato correttamente.

11. Visualizzazione della risposta

Per visualizzare la risposta di Gemini ho utilizzato:

```
print(f"AI: {response.text}\n")
```

In questo modo nel terminale viene visualizzata la risposta generata dall'intelligenza artificiale preceduta dalla scritta **AI:**.

```
43 while True:
44     domanda = input("Tu: ")
45
46     if domanda.lower() == "esci":
47         print("Arrivederci!")
48         break
49
50     response = client.models.generate_content(
51         model="gemini-3.5-flash",
52         contents=domanda,
53         config=config
54     )
55
56     print(f"AI: {response.text}\n")
```

12. Errori riscontrati

Durante lo svolgimento ho riscontrato diversi errori.

Il primo problema riguardava la variabile:

GEMINI_API_KEY

che inizialmente non veniva trovata dal programma. Il problema è stato risolto configurando correttamente il file `.env` e utilizzando `load_dotenv()`.

Successivamente ho avuto alcuni errori di sintassi e indentazione, ad esempio:

SyntaxError: unterminated string literal

Questo errore era dovuto a una stringa non chiusa correttamente.

Ho inoltre avuto:

IndentationError

perché alcune istruzioni del ciclo `while` non erano indentate correttamente.

Infine ho ricevuto:

404 NOT_FOUND

perché il modello `gemini-2.5-flash` non era disponibile per la mia API Key. Ho risolto utilizzando `gemini-3.5-flash`.

Questi errori mi hanno permesso di capire meglio l'importanza della sintassi Python, dell'indentazione e della corretta configurazione delle API.

13. Test finale

Dopo aver corretto gli errori, ho eseguito il programma dal terminale con:

```
python script.py
```

Il programma ha mostrato:

Tu:

A questo punto ho potuto inserire una domanda, ad esempio:

Tu: ciao

Il modello ha elaborato la richiesta e restituito una risposta.

Successivamente il programma è tornato nuovamente a:

Tu:

per permettere di inserire un'altra domanda.

Per terminare la conversazione ho utilizzato:

esci

ottenendo: Arrivederci!

```

└─(venv ai)-(kali@kali)-[~/Desktop/epicode/p
python_ai]
• └─$ python script.py
Tu: ciao
Direct use of automatic function calling (AFC)
in Models.generate_content is not recommended
. Instead, we recommend to use AFC in Chat.sen
d message. Similarly, direct use of AFC in Mod
els.generate_content stream is not recommended
. Instead, we recommend to use AFC in Chat.sen
d message stream.
AI: Ciao! Sono un Cyber Security Analyst.

Sono a tua disposizione per aiutarti ad analiz
zare minacce, valutare vulnerabilità, interpre
tare log di sistema o di rete, analizzare codi
ce sospetto o discutere di best practice di ha
rdening e sicurezza informatica.

Come posso aiutarti oggi?

*Nota: Se hai dati, log o porzioni di codice d
a analizzare, condividili pure, ma assicurati
di **rimuovere o mascherare qualsiasi dato sen
sibile**, come password, chiavi API, IP pubbli
ci reali o informazioni personali (PII).*

Tu: esci
Arrivederci!

└─(venv ai)-(kali@kali)-[~/Desktop/epicode/p
python_ai]
o └─$ █

```

14. Codice finale

Il codice finale utilizzato nell'esercitazione è:

```

from google import genai
from google.genai import types
import os
from dotenv import load_dotenv

load_dotenv()

api_key = os.environ["GEMINI_API_KEY"]
client = genai.Client(api_key=api_key)

# System Prompt
system_prompt = """
Sei un esperto Cyber Security Analyst.
Analizza i dati forniti e rispondi in modo tecnico ma comprensibile.
Se non sei sicuro, dillo esplicitamente.
"""

```

```

# Safety Settings
safety_settings = [
    types.SafetySetting(
        category="HARM_CATEGORY_DANGEROUS_CONTENT",
        threshold="BLOCK_NONE"
    ),
    types.SafetySetting(
        category="HARM_CATEGORY_HATE_SPEECH",
        threshold="BLOCK_NONE"
    ),
    types.SafetySetting(
        category="HARM_CATEGORY_HARASSMENT",
        threshold="BLOCK_NONE"
    ),
    types.SafetySetting(
        category="HARM_CATEGORY_SEXUALLY_EXPLICIT",
        threshold="BLOCK_NONE"
    )
]

```

```

# Configurazione
config = types.GenerateContentConfig(
    temperature=0.2,
    max_output_tokens=1024,
    system_instruction=system_prompt,
    safety_settings=safety_settings
)

```

```

# Conversazione continua
while True:
    domanda = input("Tu: ")

    if domanda.lower() == "esci":
        print("Arrivederci!")
        break

    response = client.models.generate_content(
        model="gemini-3.5-flash",
        contents=domanda,
        config=config
    )

    print(f"AI: {response.text}\n")

```

15. Conclusioni

L'esercitazione mi ha permesso di imparare come utilizzare un'API di intelligenza artificiale all'interno di un programma Python.

Ho imparato a:

- configurare una API Key tramite un file `.env`;
- creare un client Gemini;
- utilizzare un System Prompt;
- configurare la temperatura del modello;
- impostare il numero massimo di token;
- configurare i Safety Settings;
- inviare richieste al modello;
- ricevere e stampare le risposte;
- utilizzare un ciclo `while` per creare una conversazione continua;
- gestire un comando di uscita;
- individuare e correggere errori di sintassi, indentazione e configurazione del modello.

Alla fine dell'esercitazione sono riuscito a creare un piccolo chatbot da terminale basato sulle API di Gemini, specializzato nelle risposte relative alla Cyber Security.